Lonca Görev Panosu API

Proje Tanıtım Dokümanı · Node.js ile Backend Programlama — Bitirme Projesi

Hazırlayan: Doğan Can Karataş · **Kaynak kod:** github.com/dckaratas/lonca-api

1. Projenin Amacı

Bu proje; eğitim süresince öğrenilen **Node.js, Express.js, routing, middleware ve CRUD** konularını gerçek hayatta kullanılabilecek bir **görev yönetim sistemi API'si** üzerinde uygulamak amacıyla geliştirilmiştir. Sistem tamamen REST API mantığıyla çalışır; web paneli, mobil uygulama veya Postman gibi herhangi bir istemci tarafından kullanılabilir.

2. Senaryo

Yönergedeki "ekip içindeki görevleri takip etmek isteyen yazılım şirketi" senaryosu fantastik bir temaya uyarlanmıştır: **bir maceracılar loncası.** Lonca ustası şehirden gelen işleri (goblin temizliği, kayıp kervanı bulma, ejderha yuvasından hazine çalma…) görev panosuna asar; her görev bir maceracıya atanır, önceliği belirlenir ve tamamlanana kadar durumu takip edilir.

Yönergedeki kavram	Projedeki karşılığı
Şirket / ekip	Maceracılar loncası
Çalışan	Maceracı (`assignee`)
Görev	Görev panosundaki ilan (`/tasks`)
Öncelik	`priority`: critical / high / medium / low — arayüzde S / A / B / C rütbesi
Durum takibi	`status`: open → in_progress → completed

3. Sistem Özellikleri

- **Görev yönetimi (CRUD):** ekleme `POST /tasks`, listeleme `GET /tasks`, detay `GET /tasks/:id`, güncelleme `PUT /tasks/:id`, silme `DELETE /tasks/:id`
- **Logger middleware:** her isteğin method, endpoint, zaman damgası, durum kodu ve süresi konsola ve `logs/requests.log` dosyasına yazılır
- **Hata yönetimi:** geçersiz veride `400`, bulunamayan görevde `404`, beklenmeyen hatada `500`; tüm hatalar `{ "error": "..." }` biçiminde
- **Ek (isteğe bağlı pratikler):** duruma/önceliğe göre filtreleme, arama, kişiye göre listeleme, sıralama ve sayfalama; doğrulama (validation) middleware'i; `/reports` raporlama servisleri; API, test ve kullanım dokümantasyonu; 11 otomatik test

4. Veri Modeli

Alan	Tip	Açıklama
`id`	number	Sunucu tarafından otomatik atanır
`title`	string	**Zorunlu**, en fazla 100 karakter
`description`	string	Görev açıklaması
`priority`	critical | high | medium | low	Varsayılan `medium`
`status`	open | in_progress | completed	Yeni görev `open` başlar
`assignee`	string | null	Görevi üstlenen maceracı
`reward`	number	Altın ödülü (≥ 0)
`createdAt` / `updatedAt`	ISO tarih	Zaman damgaları

Veriler `src/data/tasks.json` dosyasında saklanır; sunucu yeniden başlatıldığında kaybolmaz.

5. Mimari

`İstek → express.json() → Logger → Router → (Doğrulama) → Controller → tasks.json`

Katman / Dosya	Görevi
`src/server.js`, `src/app.js`	Sunucuyu başlatır; middleware'ler, 404 ve hata yakalayıcı
`src/routes/`	Endpoint → middleware → controller eşlemesi (`/tasks`, `/reports`)
`src/controllers/`	CRUD, filtreleme ve rapor iş mantığı
`src/middleware/`	Logger ve doğrulama middleware'leri
`src/data/store.js`	JSON dosyası okuma/yazma

6. Kurulum ve Test

Node.js 18+ gerekir. `git clone https://github.com/dckaratas/lonca-api.git` → `cd lonca-api` → `npm install` → `npm start` komutlarıyla sunucu `http://localhost:3000` adresinde çalışır. Ayrıntılı adımlar repodaki **README.md** dosyasındadır.

Beş CRUD endpoint'inin tamamı **Postman** ile test edilmiştir; ekran görüntüleri ayrı dokümanda ve `docs/screenshots/` klasöründedir. Koleksiyon `docs/lonca-api.postman_collection.json` dosyasındadır. Ayrıca `npm test` ile 11 otomatik test çalışır.

Lonca Görev Panosu API — Veri Modeli Tasarımı

Hazırlayan: Doğan Can Karataş

Bu doküman, sistemin temel varlığı olan **Görev (Task)** için veri yapısının nasıl belirlendiğini açıklar. Uç noktalar ve istek/yanıt örnekleri için API Dokümantasyonu'na bakınız.

1. Varlık: Görev (Task)

Senaryodaki her ihtiyaç bir alana karşılık gelir:

İhtiyaç (yönerge)	Alan
Görev oluşturulması	`id`, `title`, `description`, `createdAt`
Görevin çalışana atanması	`assignee`
Durum takibi	`status`, `updatedAt`
Öncelik yönetimi	`priority`
Temanın gereği (lonca ödülü)	`reward`

2. Alanlar

Alan	Tip	Zorunlu	Varsayılan	Kural	Açıklama
`id`	number	—	otomatik	Pozitif tam sayı, benzersiz	En büyük id + 1 olarak sunucu atar
`title`	string	**Evet**	—	Boş olamaz, en fazla 100 karakter	Görev başlığı
`description`	string	Hayır	`""`	En fazla 1000 karakter	Görev açıklaması
`priority`	enum	Hayır	`medium`	`critical`, `high`, `medium`, `low`	Öncelik
`status`	enum	—	`open`	`open`, `in_progress`, `completed`	Durum; yeni görev her zaman `open` başlar
`assignee`	string | null	Hayır	`null`	Boş olmayan metin, en fazla 50 karakter	Görevi üstlenen maceracı
`reward`	number	Hayır	`0`	0 veya daha büyük	Altın ödülü
`createdAt`	string (ISO 8601)	—	oluşturma anı	—	Sunucu atar
`updatedAt`	string (ISO 8601)	—	—	—	Yalnızca güncellenen görevlerde bulunur

"Zorunlu" sütununda "—" olan alanlar istemciden alınmaz, sunucu tarafından belirlenir. Kurallar `src/middleware/validateTask.js` içinde uygulanır; ihlalde API `400 Bad Request` döner.

3. Sabit değer listeleri (enum)

Değerler `src/constants.js` dosyasında tek yerde tanımlıdır.

Öncelik (`priority`) — arayüzde lonca rütbesi olarak gösterilir:

Değer	Rütbe	Anlamı	Sıralama ağırlığı
`critical`	S	Kritik	4

Değer	Rütbe	Anlamı	Sıralama ağırlığı
`high`	A	Yüksek	3
`medium`	B	Orta	2
`low`	C	Düşük	1

Durum (`status`):

```
  open  ──────►  in_progress  ──────►  completed
(Açık)          (Sürüyor)            (Tamamlandı)
```

Durumlar arasında geçiş serbesttir (ör. tamamlanan bir görev yeniden açılabilir); bu, eğitim kapsamında iş kurallarını basit tutmak için bilinçli bir tercihtir.

4. Örnek kayıt

```
{
  "id": 5,
  "title": "Batık Tapınağın Lanetini Kaldır",
  "description": "Tapınaktaki lanetli totemi bul ve rahibeye teslim et.",
  "priority": "high",
  "status": "in_progress",
  "assignee": "Lyra Gecerüzgârı",
  "reward": 400,
  "createdAt": "2026-10-05T11:00:00.000Z",
  "updatedAt": "2026-10-06T09:30:00.000Z"
}
```

5. Saklama

- Görevler `src/data/tasks.json` dosyasında bir **dizi** olarak tutulur; sunucu yeniden başlasa da kaybolmaz.
- Okuma/yazma yalnızca `src/data/store.js` üzerinden yapılır (`readTasks`, `writeTasks`). Veri kaynağı değiştirilmek istenirse (ör. SQL Server) yalnızca bu dosyanın değişmesi yeterlidir.
- Başlangıçta 5 örnek görev bulunur.

6. İlişkisel karşılığı

Aynı model, **lonca-sql** projesindeki `LoncaDB` veri tabanında `Tasks` tablosu olarak da tasarlanmıştır:

JSON alanı	SQL sütunu	SQL tipi / kısıt
`id`	`TaskID`	`INT IDENTITY(1,1) PRIMARY KEY`
`title`	`Title`	`NVARCHAR(100) NOT NULL`
`description`	`Description`	`NVARCHAR(500)`
`priority`	`Priority`	`VARCHAR(10) NOT NULL DEFAULT 'medium'`, `CHECK (Priority IN ('critical','high','medium','low'))`
`status`	`Status`	`VARCHAR(20) NOT NULL DEFAULT 'open'`, `CHECK (Status IN ('open','in_progress','completed'))`
`assignee`	`AssigneeID`	`INT NULL FOREIGN KEY → Adventurers(AdventurerID)` (atanan kişi ayrı tabloda)

JSON alanı	SQL sütunu	SQL tipi / kısıt
`reward`	`RewardGold`	`INT NOT NULL DEFAULT 0`
`createdAt`	`CreatedAt`	`DATETIME NOT NULL DEFAULT GETDATE()`
`updatedAt`	`UpdatedAt`	`DATETIME NULL`

JSON modelde atanan kişi basitlik için metin olarak tutulurken, ilişkisel modelde normalizasyon gereği ayrı bir `Adventurers` tablosuna yabancı anahtarla bağlanır.

Lonca Görev Panosu API — API Tasarım Dokümanı ve Dokümantasyonu

Temel adres: `http://localhost:3000` **Veri biçimi:** İstek ve yanıt gövdeleri JSON'dur (`Content-Type: application/json`). **Kimlik doğrulama:** Yok (eğitim projesi, yerel kullanım).

İçindekiler

1. Veri modeli
2. Görev uç noktaları
3. Filtreleme, arama, sıralama ve sayfalama
4. Rapor uç noktaları
5. Doğrulama kuralları
6. Hata yanıtları

---

1. Veri modeli

```
{
  "id": 3,
  "title": "Kül Dağı Ejderhasını Uyandırma",
  "description": "Ejderhanın yuvasından Yıldız Taşı'nı fark edilmeden çal.",
  "priority": "critical",
  "status": "open",
  "assignee": null,
  "reward": 1000,
  "createdAt": "2026-10-03T08:15:00.000Z",
  "updatedAt": "2026-10-04T10:00:00.000Z"
}
```

Alan	Tip	Açıklama
`id`	number	Sunucu tarafından atanır
`title`	string	**Zorunlu**, en fazla 100 karakter
`description`	string	En fazla 1000 karakter, varsayılan `""`
`priority`	`critical` | `high` | `medium` | `low`	Varsayılan `medium`. Arayüzde S / A / B / C rütbesi
`status`	`open` | `in_progress` | `completed`	Yeni görev her zaman `open` başlar
`assignee`	string | null	Görevi üstlenen maceracı, en fazla 50 karakter
`reward`	number	Altın ödülü, 0 veya daha büyük. Varsayılan `0`
`createdAt`	ISO tarih	Oluşturulma zamanı
`updatedAt`	ISO tarih	Son güncelleme zamanı (yalnızca güncellenmiş görevlerde)

---

2. Görev uç noktaları

Method	Endpoint	Açıklama	Başarılı yanıt
GET	`/tasks`	Görevleri listele (filtrelenebilir)	200
GET	`/tasks/:id`	Görev detayı	200
POST	`/tasks`	Yeni görev ekle	201
PUT	`/tasks/:id`	Görevi güncelle	200
DELETE	`/tasks/:id`	Görevi sil	200
GET	`/tasks/search?keyword=`	Anahtar kelimeyle arama	200
GET	`/tasks/assignee/:name`	Bir maceracının görevleri	200

GET /tasks

Parametresiz çağrıldığında tüm görevleri dizi olarak döndürür.

```
GET /tasks
```

```
[
  { "id": 1, "title": "Kuzey Mağarasındaki Goblinleri Temizle", "priority": "low", "status":
"open", ... },
  { "id": 2, "title": "Kayıp Kervanı Bul", "priority": "medium", "status": "in_progress", ... }
]
```

Filtre ve sayfalama parametreleri için 3. bölüme bakınız.

GET /tasks/:id

```
GET /tasks/3
```

200 OK

```
{
  "id": 3,
  "title": "Kül Dağı Ejderhasını Uyandırma",
  "description": "Ejderhanın yuvasından Yıldız Taşı'nı fark edilmeden çal.",
  "priority": "critical",
  "status": "open",
  "assignee": null,
  "reward": 1000,
  "createdAt": "2026-10-03T08:15:00.000Z"
}
```

404 Not Found — görev yoksa: `{ "error": "Görev bulunamadı" }` **400 Bad Request** — id sayı değilse: `{ "error": "id pozitif bir tam sayı olmalıdır", "details": [...] }`

POST /tasks

Yalnızca `title` zorunludur; diğer alanlar varsayılan değerleri alır. `status` gönderilse bile yeni görev `open` başlar.

```
POST /tasks
Content-Type: application/json

{
  "title": "Kayıp Ejderha Yumurtasını Bul",
  "description": "Kül Dağı'nın eteklerinde kaybolan ejderha yumurtasını bul ve loncaya getir.",
  "priority": "high",
  "assignee": "Aria Gümüşok",
  "reward": 300
}
```

201 Created

```
{
  "id": 6,
  "title": "Kayıp Ejderha Yumurtasını Bul",
  "description": "Kül Dağı'nın eteklerinde kaybolan ejderha yumurtasını bul ve loncaya getir.",
  "priority": "high",
  "status": "open",
  "assignee": "Aria Gümüşok",
  "reward": 300,
  "createdAt": "2026-10-12T08:10:00.000Z"
}
```

PUT /tasks/:id

Kısmi güncelleme yapar: yalnızca gönderilen alanlar değişir, `updatedAt` güncellenir.

```
PUT /tasks/6
Content-Type: application/json

{
  "status": "in_progress",
  "priority": "critical"
}
```

200 OK — güncellenmiş görevin tamamı döner.

```
{
  "id": 6,
  "title": "Kayıp Ejderha Yumurtasını Bul",
  "priority": "critical",
  "status": "in_progress",
  "...": "...",
  "updatedAt": "2026-10-12T08:15:00.000Z"
}
```

DELETE /tasks/:id

```
DELETE /tasks/6
```

200 OK

```
{
  "message": "Görev silindi",
  "task": { "id": 6, "title": "Kayıp Ejderha Yumurtasını Bul", "...": "..." }
}
```

GET /tasks/search?keyword=

Anahtar kelimeyi görevin **başlığında ve açıklamasında** arar. Büyük/küçük harf duyarsızdır ve Türkçe karakterleri doğru eşler (`EJDERHA` = `ejderha`). `keyword` zorunludur.

```
GET /tasks/search?keyword=ejderha
```

200 OK

```
[
  {
    "id": 3,
    "title": "Kül Dağı Ejderhasını Uyandırma",
    "description": "Ejderhanın yuvasından Yıldız Taşı'nı fark edilmeden çal.",
    "priority": "critical",
    "status": "open",
    "assignee": null,
    "reward": 1000,
    "createdAt": "2026-10-03T08:15:00.000Z"
  }
]
```

400 Bad Request — `keyword` yoksa: `{ "error": "keyword parametresi zorunludur" }`

GET /tasks/assignee/:name

Bir maceracıya atanmış görevleri döndürür. İsim büyük/küçük harf duyarsız, tam eşleşmeyle aranır. Boşluk ve Türkçe karakterler URL'de kodlanmalıdır (Postman ve tarayıcı bunu otomatik yapar).

```
GET /tasks/assignee/Aria%20G%C3%BCm%C3%BC%C5%9Fok
```

200 OK

```
[
  {
    "id": 2,
    "title": "Kayıp Kervanı Bul",
    "priority": "medium",
    "status": "in_progress",
    "assignee": "Aria Gümüşok",
    "reward": 120,
    "...": "..."
  }
]
```

---

3. Filtreleme, arama, sıralama ve sayfalama

`GET /tasks`, `GET /tasks/search` ve `GET /tasks/assignee/:name` aşağıdaki sorgu parametrelerini kabul eder. Parametreler birlikte kullanılabilir.

Parametre	Değerler	Açıklama
`status`	`open`, `in_progress`, `completed`	Duruma göre filtre
`priority`	`critical`, `high`, `medium`, `low`	Önceliğe göre filtre
`assignee`	metin	Atanan kişiye göre filtre
`keyword`	metin	Başlık/açıklamada arama
`sort`	`createdAt`, `reward`, `priority`, `title`	Sıralama alanı
`order`	`asc` (varsayılan), `desc`	Sıralama yönü. `priority` için `desc` = en acil önce
`page`	pozitif tam sayı	Sayfa numarası (varsayılan 1)
`limit`	1–100	Sayfa başına kayıt (varsayılan 10)

Örnekler

İstek	Sonuç
`GET /tasks?status=completed`	Tamamlanan görevler
`GET /tasks?priority=high`	Yüksek öncelikli görevler
`GET /tasks?status=open&priority=critical`	Açık ve kritik görevler
`GET /tasks?sort=createdAt`	En eskiden en yeniye
`GET /tasks?sort=reward&order=desc`	En yüksek ödüllü önce
`GET /tasks?page=1&limit=10`	İlk 10 görev ve sayfa bilgisi

Sayfalı yanıt biçimi

`page` veya `limit` verilmezse yanıt düz bir dizidir (web paneli bu biçimi kullanır). Biri verilirse yanıt sayfa bilgisiyle birlikte gelir:

```
GET /tasks?sort=reward&order=desc&page=1&limit=2
```

```
{
  "data": [
    { "id": 3, "title": "Kül Dağı Ejderhasını Uyandırma", "reward": 1000, "...": "..." },
    { "id": 5, "title": "Batık Tapınağın Lanetini Kaldır", "reward": 400, "...": "..." }
  ],
  "pagination": {
    "page": 1,
    "limit": 2,
    "total": 5,
    "totalPages": 3
  }
}
```

Geçersiz bir parametre değeri `400` döner:

```
GET /tasks?status=done
```

```
{
  "error": "status şunlardan biri olmalıdır: open, in_progress, completed",
  "details": ["status şunlardan biri olmalıdır: open, in_progress, completed"]
}
```

---

4. Rapor uç noktaları

Method	Endpoint	Açıklama
GET	`/reports/completed`	Tamamlanan görevler, sayısı ve toplam ödülü
GET	`/reports/pending`	Bekleyen görevler (en acil önce), durum dağılımı, atanmamış görev sayısı
GET	`/reports/summary`	Genel özet: durum/öncelik dağılımı, tamamlanma oranı, ödüller, maceracı bazında istatistik

GET /reports/completed

```
{
  "count": 1,
  "totalReward": 30,
  "tasks": [
    { "id": 4, "title": "Simyacıya Ay Çiçeği Topla", "status": "completed", "assignee": "Borin
Demirsakal", "reward": 30, "...": "..." }
  ]
}
```

GET /reports/pending

Tamamlanmamış (`open` ve `in_progress`) görevler önceliğe göre sıralanır: `critical` → `low` .

```
{
  "count": 4,
  "byStatus": { "open": 2, "in_progress": 2 },
  "unassigned": 2,
  "totalReward": 1570,
  "tasks": [
    { "id": 3, "title": "Kül Dağı Ejderhasını Uyandırma", "priority": "critical", "...": "..." },
    { "id": 5, "title": "Batık Tapınağın Lanetini Kaldır", "priority": "high", "...": "..." },
    { "id": 2, "title": "Kayıp Kervanı Bul", "priority": "medium", "...": "..." },
    { "id": 1, "title": "Kuzey Mağarasındaki Goblinleri Temizle", "priority": "low", "...": "..."
}
  ]
}
```


GET /reports/summary


```
{
  "total": 5,
  "byStatus": { "open": 2, "in_progress": 2, "completed": 1 },
  "byPriority": { "critical": 1, "high": 1, "medium": 1, "low": 2 },
  "completionRate": 20,
  "rewards": { "total": 1600, "paid": 30, "pending": 1570 },
  "assignees": [
    { "assignee": "Aria Gümüşok", "total": 1, "completed": 0, "earnedReward": 0 },
    { "assignee": "Borin Demirsakal", "total": 1, "completed": 1, "earnedReward": 30 },
    { "assignee": "Lyra Gecerüzgârı", "total": 1, "completed": 0, "earnedReward": 0 }
  ]
}
```


Alan	Açıklama
`completionRate`	Tamamlanan görevlerin yüzdesi (tam sayıya yuvarlanır)
`rewards.paid`	Tamamlanan görevlerin ödül toplamı
`rewards.pending`	Henüz tamamlanmamış görevlerin ödül toplamı
`assignees[].earnedReward`	Maceracının tamamladığı görevlerden kazandığı altın

---

5. Doğrulama kuralları

Doğrulama `src/middleware/validateTask.js` içindeki middleware'lerle, istek controller'a ulaşmadan yapılır.

Middleware	Nerede	Kontrol ettiği
`validateCreate`	`POST /tasks`	Gövde alanları; `title` zorunlu
`validateUpdate`	`PUT /tasks/:id`	Gövde alanları; gönderilmeyenler atlanır, `status` kontrol edilir
`validateId`	`:id` içeren tüm istekler	`id` pozitif tam sayı
`validateQuery`	Listeleme istekleri	Filtre, sıralama ve sayfalama parametreleri

Alan	Kural	Hata mesajı
`title`	Zorunlu (POST), boş olmayan metin, ≤ 100 karakter	`title alanı zorunludur` / `title boş olmayan bir metin olmalıdır`
`description`	Metin, ≤ 1000 karakter	`description metin olmalıdır`
`priority`	`critical`, `high`, `medium`, `low`	`priority şunlardan biri olmalıdır: ...`
`status`	`open`, `in_progress`, `completed` (PUT)	`status şunlardan biri olmalıdır: ...`
`assignee`	Boş olmayan metin veya `null`, ≤ 50 karakter	`assignee boş olmayan bir metin veya null olmalıdır`
`reward`	Sayı, ≥ 0	`reward 0 veya daha büyük bir sayı olmalıdır`

Birden fazla hata varsa hepsi `details` dizisinde listelenir; `error` ilk hatayı içerir:

```
POST /tasks
Content-Type: application/json

{ "priority": "urgent", "reward": -10, "assignee": "" }
```

400 Bad Request

```
{
  "error": "title alanı zorunludur",
  "details": [
    "title alanı zorunludur",
    "priority şunlardan biri olmalıdır: critical, high, medium, low",
    "assignee boş olmayan bir metin veya null olmalıdır",
    "reward 0 veya daha büyük bir sayı olmalıdır"
  ]
}
```

---

6. Hata yanıtları

Tüm hatalar `{ "error": "..." }` biçimindedir; doğrulama hatalarında ek olarak `details` dizisi bulunur.

Kod	Ne zaman	Örnek
400	Geçersiz görev verisi, id veya sorgu parametresi	`{ "error": "title alanı zorunludur", "details": [...] }`
400	Bozuk JSON gövdesi	`{ "error": "Geçersiz JSON gövdesi" }`
404	Görev bulunamadı	`{ "error": "Görev bulunamadı" }`
404	Tanımsız endpoint	`{ "error": "Bulunamadı: GET /quests" }`
413	Gövde 10 KB'den büyük	`{ "error": "İstek gövdesi çok büyük" }`
500	Beklenmeyen sunucu hatası	`{ "error": "Sunucu hatası" }`